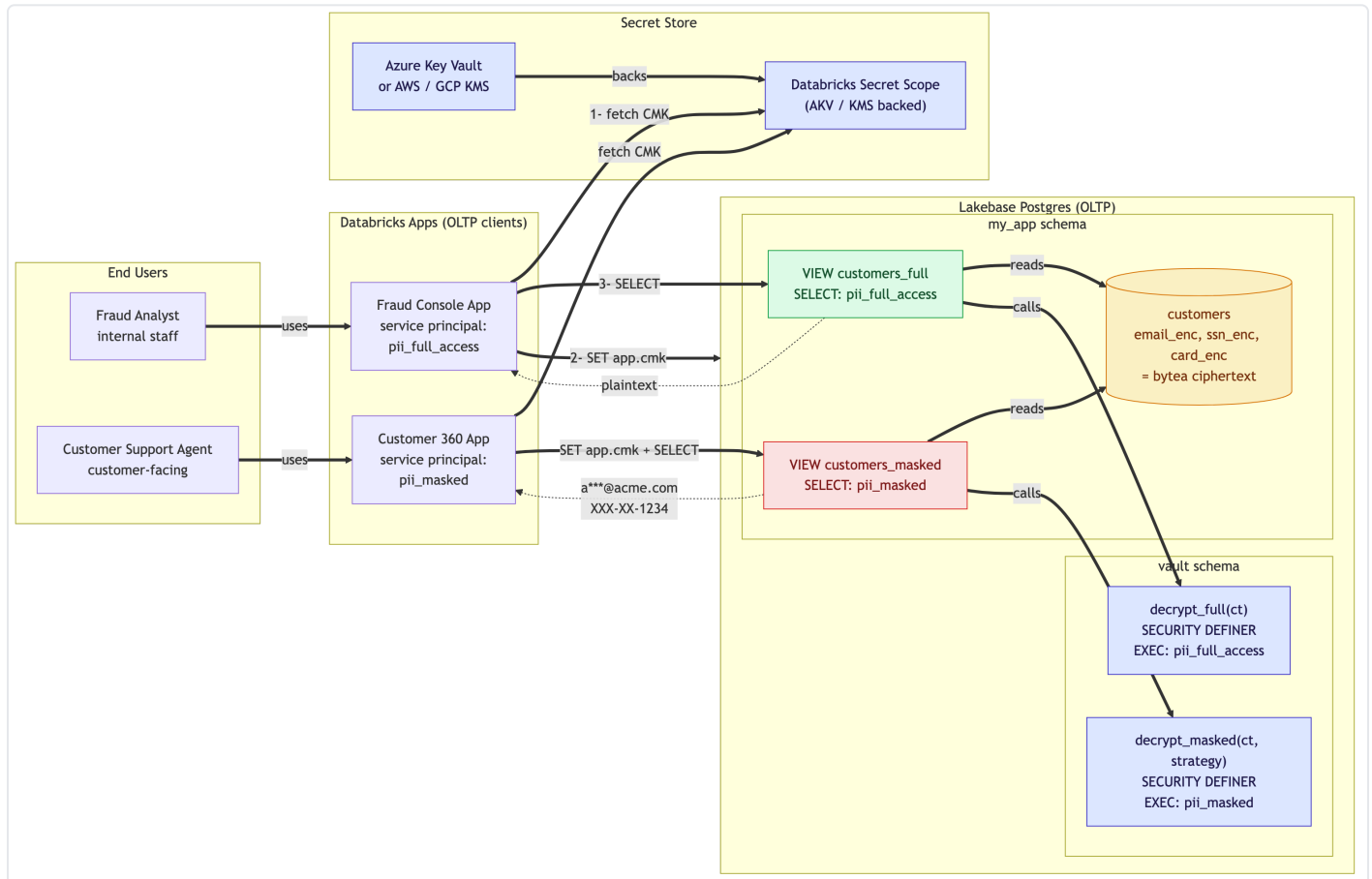


Field-Level Encryption & Dynamic Masking

Use case: Protect PII in OLTP Postgres
Components: Lakebase · pgcrypto · AKV / KMS · Databricks Apps

Encryption-at-rest only protects you against a stolen disk. Once a user authenticates, they see plaintext. **Field-level encryption** stores PII as ciphertext in the table and decrypts only inside an authorized function call, so a **customer-managed key (CMK)** from **Azure Key Vault / KMS** controls who sees what. Add **role-based masking** and the same row looks completely different to a fraud analyst vs. a support agent.



CMK lives in Azure Key Vault → fetched at session start → injected into Lakebase as a session GUC → decrypt path is gated per Postgres role

1. One-time setup (Lakebase SQL Editor) 2. Table, role-specific views, and consuming app

ENCRYPTED BASE TABLE + TWO VIEWS

ENABLE PGCRYPTO, CREATE VAULT & APP SCHEMAS

```
CREATE EXTENSION IF NOT EXISTS
pgcrypto;
CREATE SCHEMA IF NOT EXISTS
my_vault;
CREATE SCHEMA IF NOT EXISTS
my_app;
```

TWO SECURITY DEFINER DECRYPT FUNCTIONS

CMK is read from `current_setting('app.cmk')`, which the app sets per session from AKV. The CMK never persists in Postgres.

```
CREATE OR REPLACE FUNCTION
my_vault.decrypt_full(p_ct
bytea)
RETURNS text LANGUAGE plpgsql
SECURITY DEFINER STABLE AS $$
BEGIN
    RETURN
public.pgp_sym_decrypt(p_ct,
current_setting('app.cmk'));
END; $$;

CREATE OR REPLACE FUNCTION
my_vault.decrypt_masked(p_ct
bytea, strategy text)
RETURNS text LANGUAGE plpgsql
SECURITY DEFINER STABLE AS $$
DECLARE v text;
BEGIN
    v :=
public.pgp_sym_decrypt(p_ct,
current_setting('app.cmk'));
    RETURN CASE strategy
        WHEN 'email' THEN
regex_replace(v, '(^).*
(.*$)', '\1***\2')
        WHEN 'ssn' THEN 'XXX-
XX-' || right(v, 4)
        WHEN 'card' THEN
'*****' || right(v, 4)
        ELSE '***REDACTED***'
    END;
END; $$;
```

ROLES & ACLS (LOCK DOWN EACH PATH)

```
CREATE ROLE pii_full_access;
CREATE ROLE pii_masked;
REVOKE EXECUTE ON FUNCTION
my_vault.decrypt_full(bytea)
FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION
my_vault.decrypt_masked(bytea,
text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
my_vault.decrypt_full(bytea)
TO pii_full_access;
GRANT EXECUTE ON FUNCTION
my_vault.decrypt_masked(bytea,
text) TO pii_masked;
```

```
CREATE TABLE my_app.customers (
    id          bigserial PRIMARY KEY,
    full_name   text,
    email_enc   bytea NOT NULL,    -- ciphertext on disk
    ssn_enc     bytea NOT NULL,
    card_number_enc bytea NOT NULL
);

-- Insert: encrypt with the session CMK
INSERT INTO my_app.customers(full_name, email_enc, ssn_enc, card_number_enc)
VALUES ('Alice',
        pgp_sym_encrypt('alice@acme.com',    current_setting('app.cmk')),
        pgp_sym_encrypt('123-45-6789',      current_setting('app.cmk')),
        pgp_sym_encrypt('4111111111111111', current_setting('app.cmk')));

CREATE VIEW my_app.customers_full AS
SELECT id, full_name,
        my_vault.decrypt_full(email_enc)      AS email,
        my_vault.decrypt_full(ssn_enc)        AS ssn,
        my_vault.decrypt_full(card_number_enc) AS card_number
FROM my_app.customers;

CREATE VIEW my_app.customers_masked AS
SELECT id, full_name,
        my_vault.decrypt_masked(email_enc,    'email') AS email,
        my_vault.decrypt_masked(ssn_enc,      'ssn')   AS ssn,
        my_vault.decrypt_masked(card_number_enc, 'card') AS card_number
FROM my_app.customers;

GRANT SELECT ON my_app.customers_full TO pii_full_access;
GRANT SELECT ON my_app.customers_masked TO pii_masked;
```

APPLICATION BOOTSTRAP (PYTHON · DATABRICKS APP)

```
import pycpg
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
cmk = w.dbutils.secrets.get(scope="lakebase-cmk", key="lakebase-cmk")
ep =
w.postgres.get_endpoint(name="projects/<p>/branches/production/endpoints/primary")
tok = w.postgres.generate_database_credential(endpoint=ep.name).token

with pycpg.connect(host=ep.status.hosts.host, dbname="databricks_postgres",
                    user=w.current_user.me().user_name, password=tok,
                    sslmode="require") as c, c.cursor() as cur:
    cur.execute("SELECT set_config('app.cmk', %s, false);", (cmk,))
    cur.execute("SET ROLE demo_pii_user;")           # or demo_masked_user
    cur.execute("SELECT * FROM my_app.customers_full;")
    print(cur.fetchall())
```

SAMPLE OUTPUT — SAME ROW, DIFFERENT ROLES

role **pii_full_access** reading my_app.customers_full

```
1 | Alice | alice@acme.com | 123-45-6789 | 4111111111111111
```

role **pii_masked** reading my_app.customers_masked

```
1 | Alice | a***@acme.com | XXX-XX-6789 | *****1111
```

Why this is different from encryption-at-rest

Threat	Encryption at rest	Field-level encryption + masking
Stolen backup / disk image	Protected	Protected
Compromised app credential	Plaintext exposed	Mask or NULL only
DBA reading rows	Plaintext exposed	Sees ciphertext only
Customer-facing app role compromised	Plaintext exposed	Masked values only

Threat	Encryption at rest	Field-level encryption + masking
Key rotation	Re-encrypts whole volume	One UPDATE re-encrypts rows in place
Audit granularity	Per-disk	Per-column · per-role · per-call

Requirements: Lakebase Autoscaling (PG 16/17) · `pgcrypto` (already on Lakebase's allowlist) · Databricks Secret Scope backed by Azure Key Vault / AWS KMS / GCP KMS.

Full repo (SQL · Python demo · diagrams): github.com/nan-databricks/lakebase-field-level-encryption

Reference pattern · Field Engineering · Databricks